

Assignment 2

Basic Info:

I used Copilot and Claude to generate the code. I accessed Claude on my computer and Copilot on google.

Prompt:

The program will prioritize emails for a busy company CEO. • You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status. • You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO. • You must create functions from scratch. Do not include pre-existing heap modules. • Test File format: Command Description EMAIL <sender category>, <subject line>, <date> The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY NEXT Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date: READ CEO has read email and dealt with it COUNT Display current count of unread emails • The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it). • Sample Test File: EMAIL Peer,Can you help me on this?,12-01-2024 EMAIL OtherPerson,Try our product,12-19-2024 EMAIL Boss,Important,12-20-2024 EMAIL Subordinate,How do I handle this?,12-25-2024 EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024 EMAIL Boss,Never Mind,01-03-2025 COUNT NEXT READ NEXT READ COUNT • Your program should display this: There are 6 emails to read. Next email: Sender: Boss Subject: Never Mind Date: 01-03-2025 Next email: Sender: Boss Subject: Important Date: 12-20-2024 There are 4 emails to read.

Generated code is on Git Hub copy and pasted exactly as it was made.

Analysis:

In terms of correctness, both were able to correctly run. They showed the correct priority of emails and showed the correct number of unread emails. Copilot has better execution time as it uses less while and for loops. When it comes to space complexity Copilot wins again. It uses only one starting variable while Copilot uses 4. Claude also has many different loops running while Copilot uses far less. However, Claude has better maintainability. It has more comments in the code and even includes commands you can copy and paste to run the program. Making it very easy for users to know how to run the code and what each line does. Adjusting the code in the future would be easier. Copilot does have a few comments, but they mostly just outline what certain blocks of code are used in and not exactly what each line does.

I will use Copilot as its better optimized and requires only a few edits. It is already simpler than Claudes code and has more comments so it's easier to understand. The code handles all cases correctly and uses its time and space efficiently. I will add comments to the code explaining how it works as well as commands to input for ease of user. This way if others want to work on it they know exactly how it was intended to run.